

Lab 04 - Stream Ciphers and Linear Feedback Shift Registers

One of the basic building blocks (the primitives) of stream ciphers that are implemented in hardware is a Feedback Shift Register. In a current stream cipher such as Grain-128a both Linear Feedback Shift Registers (LFSR) and Non-linear Feedback Shift Registers (NFSR) are implemented to provide resilience against cryptanalysis.

Using only a LFSR for encryption is a bad idea because these LFSRs can be reduced to mathematical equations which can be quickly solved. Solving those equations provides the initial register settings to be determined, allowing for the entire keystream to be generated, XORed with the ciphertext, and the plaintext message recovered. The initial settings are considered secret.

In Part 01 of this week's lab you will first encrypt a plaintext message using an LFSR. In Part 02 you will use cryptanalysis to find the initial register settings (seed) for the LFSR. Using the seed you will be able to generate the entire keystream and decrypt the message.

Part 01 - LFSR Encryption

You will be implementing an LFSR cryptosystem in python. Below is an **EXAMPLE** that walks you through the steps you need to take and indicates what you need to code. **The ACTUAL plaintext to encrypt for lab is in the file part01_plaintext.txt.**

1. Copy the skeleton code from repo.homework.3310.com using the repo/repo credentials. Remember you will have to give yourself permissions on the directory and files once they are downloaded.

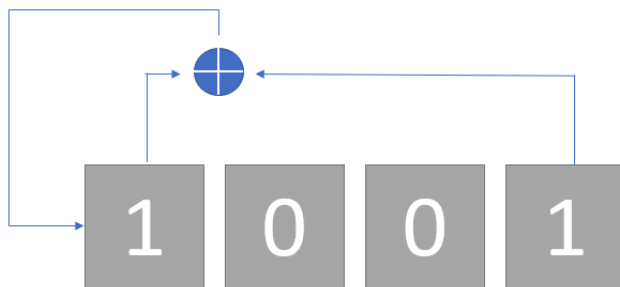
```
cpre3310@cpre331:~$ scp -r repo@repo.homework.3310.com:/home/repo/lab04 .
```

2. When we were working with classic ciphers, we would remove spaces and capitalize all characters. However, when working with symmetric ciphers you leave the capitalization and punctuation in the message you wish to encrypt. **EXAMPLE plaintext message:** *Fear is the path to the dark side...fear leads to anger...anger leads to hate...hate leads to suffering.* You have been given this text in the *part01_plaintextDemo.txt* to test that your functions are working correctly. You also will have **part01_plaintext.txt** which is the real text for this lab.
3. The functions used in part01 will be reused in part02 so there is only one piece of skeleton code. Open `part01_02_skel.py`. In your program you have three options from which the user selects:

1) encrypt/decrypt

- 2) check encryption
 - 3) conduct cryptanalysis
4. You have been given the `readFile()` function
- a. It requires each file to be identified as ascii or binary on the command line.
 - i. The plaintext file and the output text file (solution in part 02) will be ascii files
 - ii. The ciphertext file and the keystream files will be binary files
 - b. The file is read in as a single string
 - c. It converts the characters to their ASCII values and stores them in a numpy array
 - d. Then it unpacks the ASCII characters into bits using **little endian ordering**
 - e. Returns the numpy array of bits
5. You have also been given the `writeFile()` function
- a. Again, the output file must be identified as ascii or binary from the command line.
 - i. The plaintext file and the output text file (solution in part 02) will be ascii files
 - ii. The ciphertext file and the keystream files will be binary files
 - b. Every 8 bits is packed into a byte and then written out to the file
 - c. Again, **little endian ordering** is used.
6. You will use the `cycle()` function for both parts 01 and 02. For part01 you will encrypt the message with a 4 bit LFSR that uses the following taps to XOR position 0 and 3. The initial seed value is 1001. The operation is shown in the image below. This is very similar to what was demonstrated in lecture.

Look for the TODO: section in the `cycle()` function. You must modify the `cycle()` function to XOR the first and last bits (positions 0 and 3). Numpy provides a function called `numpy.bitwise_xor()` which will perform the bitwise XOR of two arrays element-wise. Please ignore the TODO: for part02 until you get to part02 in the lab document.



7. Looking a little farther down in the `cycle()` function you will find another TODO: section. In this section you will drop the Most Significant Bit (MSB) from the register, shift all other bits to a more significant bit position, and insert the XOR value in the Least Significant Bit (LSB) position. Because we are working with little endian, the **LSB is in the location at index [0] of the numpy array**.
8. The `makeKeyStream()` function generates a key stream of a given length. The length is generally the length of the plaintext or the ciphertext being encrypted or decrypted. Again, look for the TODO: section.
 - a. The key stream is generated by using the MSB at `register[3]` each time the register cycles. In this function `register[3]` is `seedValue[3]`.
 - b. Loop through the length of the plaintext or ciphertext
 - i. On the *i*th iteration you must copy the bit at `register[3]` to the keystream array at `keystream[i]`
 - ii. On the *i*+1 iteration and forward you must first cycle the register (call the `cycle()` function) and then copy the bit at `register[3]` to the keystream array
9. Finally, the `XOR()` function will again use the `numpy.bitwise_xor`
 - a. There is a variable `minLength` that is already defined for you. It allows you to XOR two lists when the length of the two inputs are not equal. It will help with the cryptanalysis section in part02 where the `knownPlaintext` and `ciphertext` will not be the same length.
 - b. Your TODO: section requires you to use the `numpy.bitwise_xor` function to XOR all the bits in the `inputData` array and the `keyStream` array from 0 to the `minLength`.
10. Now you are ready to run the program. Again, this document is using the *part01_plaintextDemo.txt* to test that your functions are working correctly. You will use **part01_plaintext.txt** as the **ACTUAL** plaintext for this lab.

```
python3 part01_02_skel.py part01_plaintext.txt ascii
part01_ciphertext.bin binary part01_keystream.bin binary
```

11. Select option 1 encrypt. You will be prompted for the seed values. For this example, the starting values are 1001.

```
Enter starting values: 1001
```

12. Because we are working with little endian, you will need to answer `y` to the endianness question. (You will need to add the code for this question on your own. Should be one line asking for input and setting it to the `convertLittle` variable)

```
Do you need to convert to little endian? y/n
```

```
y
```

13. I have printed the plaintext bits out to help you visualize what the code does. If you are running the **plaintextDemo.txt** file you should end up with the following bits printed using little endian. Since we are working with little endian, and if you want to spot check your conversion visually, you can reverse the bits in the 8-bit byte. 01100010 is

01000110 in big endian which is 70 in decimal which is ASCII for F. You should be able to do this for each character in the plaintext if you like.

```
inputDataInBits:
0110001010100110100001100100111000000
1001001011011001110000001000010111000
0101101010011000000100000011101000011
0001011100001011000000100001011101111
0110000001000010111000010110101001100
0000100001001101000011001001110110101
1000000100110011101001011000100110101
0011001110100011101000111010001100110
1010011010000110010011100000010000110
1101010011010000110001001101100111000
0001000010111011110110000001001000011
0011101101110011010100110010011100111
0100011101000111010010000110011101101
1100110101001100100111000000100001101
1010100110100001100010011011001110000
0010000101110111101100000010000010110
1000011000101110101001100111010001110
1000111010000010110100001100010111010
1001100000010000110110101001101000011
0001001101100111000000100001011101111
0110000001001100111010101110011001100
1100110101001100100111010010110011101
10111001100111010001010000
```

14. Again, I printed the keyStream bits for part 01. If you use the `plaintextDemo.txt` file and print the keyStream it should match the output below.

```
keystreamBits:
1001000111101011001000111101011001000
1111010110010001111010110010001111010
1100100011110101100100011110101100100
0111101011001000111101011001000111101
0110010001111010110010001111010110010
0011110101100100011110101100100011110
1011001000111101011001000111101011001
0001111010110010001111010110010001111
0101100100011110101100100011110101100
1000111101011001000111101011001000111
1010110010001111010110010001111010110
0100011110101100100011110101100100011
1101011001000111101011001000111101011
0010001111010110010001111010110010001
1110101100100011110101100100011110101
1001000111101011001000111101011001000
```

```
1111010110010001111010110010001111010
1100100011110101100100011110101100100
0111101011001000111101011001000111101
0110010001111010110010001111010110010
0011110101100100011110101100100011110
1011001000111101011001000111101011001
00011110101100100011110101
```

15. Finally, the result of XORing the plaintext bits and the keystream bits is printed. In this case, it is the ciphertext you are creating. Again, if you use the demo file it should match the bits below.

```
resultXOR:
1111001101001101101001011001100001000
0110011101001000001010111010011000010
1001001001101101100000011101000100111
0110110111000011111101111000011010010
0000010000111000001010011001111111110
0011010100101001011101100101010101011
0011001100001110110000011111001101100
0010110100010001100111101100011101001
1111111110011000111111000011100101010
0101101110001001110110100110101111111
1011110000110100100000010000110010101
0111110011011111110111000111111000100
1001000100000000111001001110100000110
1110111010011010110110111010010011100
0100001010000010010111111111010000101
1011000010011100100100111111011011110
0111001110111111010011010101011110100
0100011001110111010000010010111011110
1110001011011000001011110000101111110
0111011100011101110010101110001011101
0101110100101000100100000010111010010
0111111101110001111110010101011000100
101001110010111110010100101
```

16. So, let's check your work. Since the keystream is saved as `keystream.bin` you can rerun the program to decrypt the `ciphertext.bin` file.

```
python3 part01_02_skel.py part01_ciphertext.bin binary
part01_decrypted_plaintext.txt ascii part01_keystream.bin
binary
```

17. Select option 2 check encryption

18. Open the file `part01_decrypted_plaintext.txt` with a text editor. The plaintext

should drop out again if your code is correct and you are reading/writing to the ascii/binary files correctly.

Fear is the path to the dark side...fear leads to anger...anger leads to hate...hate leads to suffering.

The **ACTUAL plaintext** message to encrypt for your lab report is in the file **part01_plaintext.txt**.

Part 02 - LFSR Cryptanalysis and Decryption

You will conduct cryptanalysis on the LSFR cryptosystem using a known plaintext attack. The **example** below will walk you through the process. **The files to use for your lab report are part02_ciphertext.bin and part02_knownPlaintext.txt.** Below is just an example.

EXAMPLE ONLY:

The ciphertext was intercepted via communications from the engineers working for the Empire. We know that these messages always begin with the words, "FOR THE EMPIRE", and this known plaintext will help aid us in breaking the encryption.

1. The first step is to generate the Suspected Keystream (Potential Keystream) bits. You do that by XORing the known plaintext bits with the ciphertext bits. This has been written for you as option 3 in `part01_02_skel.py`.

```
python3 part01_02_skel.py part02_knownPlaintext.txt ascii  
part02_ciphertext.bin binary part02_possible_keyStream.bin  
binary
```
2. Select option 3 conduct cryptanalysis.
3. The resulting output is the potential Keystream. It will print out Suspected Keystream bits (potentialKeyStream in the code), as well as saving it to a file. You will want to use it as demonstrated below to calculate the period.
4. Calculate the period. To calculate the period, we must look for a pattern of repeating bits in the suspected Keystream.

Suspected Keystream

```
100100011110101100100011110101100100011110101100100011110101100  
1000111101011001000111101011001000111101011001000
```

We can see that the fifteen bits “100100011110101” repeat over and over. This means our period is fifteen bits

5. Calculate the degree of LFSR. The degree will tell us how many registers were used to encrypt the plaintext. Remember from lecture that

$$\text{Period} = 2^n - 1$$

Therefore, the degree is calculated by

$$\text{Degree} = \log_2 (\text{period} + 1)$$

In this case the degree is 4, meaning that the ciphertext was encrypted using four registers.

6. Calculate the seed. Copy the first fifteen bits from the period calculated in step 3 and line them vertically in a spreadsheet. **NOTE:** You are calculating the seed in little endian. To successfully decrypt the ciphertext later in the lab, you will need to *answer n for No to the prompt about converting the register to little endian*. I will remind you of this again when you get to that step.

	A	B	C	D	E
1					1
2					0
3					0
4					1
5					0
6					0
7					0
8					1
9					1
10					1
11					1
12					0
13					1
14					0
15					1
16					

Because of how LFSR works, you can work your output back each step until you find the seed used to generate the keystream. In this case, we know that the first four values in the registers are “1001”, so the seed is “1001”

	A	B	C	D	E
1	1	0	0	1	
2	0	1	0	0	
3	0	0	1	0	
4	0	0	0	1	
5	1	0	0	0	
6	1	1	0	0	
7	1	1	1	0	
8	1	1	1	1	
9	0	1	1	1	
10	1	0	1	1	
11	0	1	0	1	
12	1	0	1	0	
13		1	0	1	
14			1	0	
15				1	
16					

7. Identify the LFSR configuration. We know the seed, now we need to figure out the actual hardware configuration. Use the spreadsheet generated in the previous step to help this process. **We will assume there are only two registers being XORd with each other to achieve a maximal period.**

Analyzing Row 2:

We know that in Row 2, column A, the value is "0". This value has to come from two registers in Row 1 being XORd with each other. There are only two ways this can happen.

1.) Registers A and D

	A	B	C	D	E
1	1	0	0	1	
2	0	1	0	0	
3	0	0	1	0	
4	0	0	0	1	

2.) Registers B and C

	A	B	C	D	E
1	1	0	0	1	
2	0	1	0	0	
3	0	0	1	0	
4	0	0	0	1	

Analyzing Row 3:

We know that in Row 3, column A, the value is "0". From analyzing Row2 we know that this value must come from XORing Registers A and D, or from Registers B and C.

1.) Registers A and D

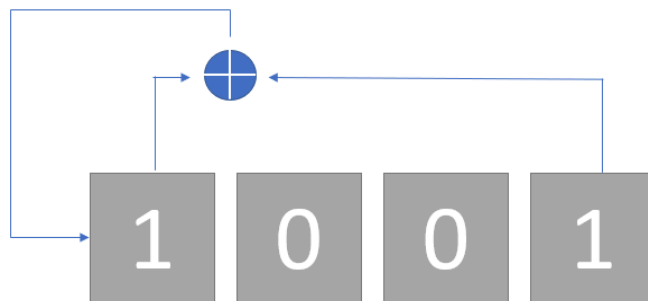
	A	B	C	D	E
1	1	0	0	1	
2	0	1	0	0	
3	0	0	1	0	
4	0	0	0	1	

2.) Registers B and C

	A	B	C	D	E
1	1	0	0	1	
2	0	1	0	0	
3	0	0	1	0	
4	0	0	0	1	

A and D is the only option that works. Again, this is for this example. You may have to work through a few more rows in the actual lab assignment.

Therefore, the LFSR looks like the one depicted below:



- Now that we know the LFSR configuration and the seed, we can calculate the keystream that was used to encrypt the plaintext. In this example the `cycle()` function is the same for part02 as it was for part01. In the cryptanalysis for the ACTUAL ciphertext, you will

need to **change the bits in the cycle() function that are being XOR'ed**. This is marked as TODO: part 02 in the cycle() function.

- a. **Comment** the original lines for part01 in the cycle () function,
- b. Change the bits to the ones that have taps based upon what you have found in steps 5 & 6. **You need the logic for both part01 and part02 to earn full credit for your program.**

9. Run your program

```
python3 part01_02_skel.py part02_ciphertext.bin binary
part02_decrypted_plaintext.txt ascii part02_keyStream.bin
binary
```

10. Select option 1 to encrypt/decrypt.

11. Enter your seed values (register). For the demo, these are still 1001. **These will be different for your actual lab assignment.**

Enter starting values: 1001

19. Because you found the seed using little endian, you will need to answer n to the endianness question.

Do you need to convert to little endian? y/n

n

12. If you are using the demo ciphertext and you correctly generated the keystream the output bits should be as follows

```
inputDataInBits:
1111001100011001011010011101001001101
1011011111000101101010111011011110000
000000001101111110110001100000110101
0101010011001010111001001100000010101
0100011011011011111010001011100100110
0110100001100000111011101100000110100
0010111010110101110011001110100001110
0000100100100111001010110011010001011
1110101110011100001000000100111101001
110110110001011101001110111110001100
1010011000000111010010110001011000011
0100010110100010101000010101111100011
0001110100000001011010010100111110011
0011101111011110101000101110100010111
0110000110100010111110101111101100110
000000001111111001101110000011001100
01110111111000111100100100100111100111
10011010101100111110101111010101110101
0111000001000000111001010011101101100
0011110101111111101100011101000011010
0001100111001101010110011110000101010
```

```
1011101111011001101000001011111010101
10111010001011101001010111
```

keystreamBits:

```
1001000111101011001000111101011001000
1111010110010001111010110010001111010
1100100011110101100100011110101100100
0111101011001000111101011001000111101
0110010001111010110010001111010110010
0011110101100100011110101100100011110
1011001000111101011001000111101011001
0001111010110010001111010110010001111
0101100100011110101100100011110101100
1000111101011001000111101011001000111
1010110010001111010110010001111010110
0100011110101100100011110101100100011
1101011001000111101011001000111101011
0010001111010110010001111010110010001
1110101100100011110101100100011110101
1001000111101011001000111101011001000
1111010110010001111010110010001111010
1100100011110101100100011110101100100
0111101011001000111101011001000111101
0110010001111010110010001111010110010
0011110101100100011110101100100011110
1011001000111101011001000111101011001
00011110101100100011110101
```

resultXOR:

```
0110001011110010010010100000010000101
0100001001010100010000001001010001010
1100100000101010010010010010101010001
0010111000000010000100010101000101000
0010001010100001001000000100110010100
0101010100000100100101000000100101010
1001110010001000101010001001001010111
0001011110010101000101100101000000100
1011001010000010100100100111001000101
0101010001001110010100000100111001011
0000101010001000000100100000100010101
0000001000001110001011100000011000000
1100101101000110110001011100000011000
0001100000001000111001010100010000110
1000101010000001001011001011110010011
1001000100010100000101001101000000100
1000001001110010001000100000010011101
```

```
0101001001000110010001100100000010001
0000101010001000000100001010101010001
0101100100000101011110010010010101000
0010010010101001001000110010100110100
0000100111100100110001001100010001100
10100100100111001010100010
```

13. Open the `part02_decrypted_plaintext.txt` file and it should contain.

```
FOR THE EMPIRE: DEATH STAR UNDERGOES MAINTENANCE AT 8:00-6:00
NEXT MONDAY AND WILL BE TEMPORARILY OFFLINE
```

Lab 04 Template

Part 01

1. **Screenshot of encrypted message from `part01_plaintext.txt` by using a 4-bit LFSR with the seed “1001” with the first and last bit XORd with each other. Use `xxd -b part01_ciphertext.bin` to display both the binary and ASCII.**
(10 points)

Part 02

1. **XOR the known plaintext with the first X bits of the ciphertext, where X is the number of bits in the known plaintext. Print out the 0s and 1s.**
(5 points)
2. **Calculate the period by showing the number of bits that are repeated in the keystream.**
(5 points)
3. **Calculate the degree (Show your work step by step).**
(5 points)
4. **Calculate the seed by pasting your keystream into a spreadsheet and tracing it back to the initial starting values.**
(10 points)
5. **Use the spreadsheet from step 4 to find the LFSR configuration and show your work for each step.**
(10 points)

- 6. Draw a diagram of the LFSR configuration you identified in step 5.**
(10 points)
- 7. Translate the plaintext into ASCII.**
(15 points)
- 8. Upload your code to canvas as "<netid>_lab04_part01_02.py". A TA will run your program and make sure it can decrypt the ciphertext. Double check that it runs without any issues before uploading it. Be sure to have both lines in the cycle() to demonstrate the part01 bits and the part02 bits.**
(30 points)